

Quantum Computing and Quantum Machine Learning

Morten Hjorth-Jensen¹

Department of Physics, University of Oslo¹

April 29, 2026

© 1999-2026, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plan for the week of April 27-May 2

1. Discussion of the QAOA algorithm with repetition from last week
2. Parametrized quantum circuits (PQC) and Variational Quantum Circuits (VQCs)
3. Quantum neural networks (QNNs)
 - ▶ Training QNNs and Loss Landscapes

What is Quantum Machine Learning?

Quantum Machine Learning (QML) integrates quantum computing with machine learning algorithms to exploit quantum advantages. It explores how quantum computing can enhance classical machine learning.

Motivation:

1. High-dimensional Hilbert spaces for better feature representation.
2. Quantum parallelism for faster computation.
3. Quantum entanglement for richer data encoding.

Quantum Speedups in ML

Why Quantum?

1. **Quantum Parallelism:** Process multiple states simultaneously.
2. **Quantum Entanglement:** Correlated states for richer information.
3. **Quantum Interference:** Constructive and destructive interference to enhance solutions.

Challenges in Quantum Machine Learning

Quantum Hardware Limitations:

1. Noisy Intermediate-Scale Quantum (NISQ) devices.
2. Decoherence and limited qubit coherence times.

Data Encoding:

1. Efficient embedding of classical data into quantum states.

Scalability:

1. Difficult to scale circuits to large datasets.

Quantum neural network

Another variation is the quantum variational classifier, sometimes called a quantum neural network (to be discussed below). Instead of precomputing a fixed kernel, one trains a parameterized quantum circuit to output labels. Interestingly, Schuld (2021) shows that variational quantum models, when trained by minimizing a loss, are mathematically equivalent to kernel machines with a particular kernel determined by the circuit. In fact, one can often find a kernel SVM that matches or outperforms the variational model. In practice, one can combine these: use a trainable quantum embedding $U(\mathbf{x}; \Theta)$ with tunable parameters Θ , and optimize Θ to maximize the SVM classification accuracy. This is called a quantum kernel learning approach.

Quantum Neural Networks and Variational Circuits

The Variational Quantum Algorithm (VQA) is a Variational Quantum Circuit (VQC), that is a quantum circuit with tunable parameters and which is trained using a classical optimizer. In practice, a VQC (also called a Parameterized Quantum Circuit (PQC)) is used as a Quantum Neural Network (QNN): data are encoded into quantum states, a parameterized circuit is applied, and measurements yield outputs. For example, Abbas et al. showed that certain QNNs can exhibit higher effective dimension (and thus capacity to generalize) than comparable classical networks , suggesting a potential quantum advantage.

Below we develop the mathematical foundations (state preparation, parameterized unitaries, measurement), discuss optimization and training challenges, and work through practical code examples using PennyLane.

Variational Quantum Circuits

Variational Quantum Algorithms (VQAs) are hybrid schemes where a quantum circuit with adjustable parameters is trained by a classical optimizer . In this framework, a Variational Quantum Circuit (VQC) typically has three parts : (i) a state preparation or feature map that encodes classical input x into a quantum state; (ii) a parameterized circuit $W(\Theta)$ (often called the ansatz) that depends on trainable parameters Θ ; and (iii) a measurement that extracts a classical output from the final quantum state.

Setting up a VQC

Given an input vector $\mathbf{x} = (x_1, \dots, x_n)$, we prepare the initial state

$$|\psi_{\text{in}}\rangle = U(\mathbf{x})|0\rangle^{\otimes n},$$

where $U(\mathbf{x})$ is a unitary (possibly composed of rotations) that depends on the data. We then apply the variational circuit $W(\Theta)$, often built as a product of layers $V_j(\Theta_j)$, so that the final state is

$$|\Psi(\mathbf{x}; \Theta)\rangle = W(\Theta)U(\mathbf{x})|0\rangle^{\otimes n}.$$

For instance, one common ansatz is the hardware-efficient circuit: layers of parameterized single-qubit rotations and entangling gates (like CNOTs) repeated several times. The structure of $W(\Theta)$ can dramatically affect the circuit's expressivity and trainability.

Outputs

To produce a scalar or vector output, we measure one or more observables $\hat{B}_k$ on the final state. The network's output is given by the expectation values:

$$f_k(x; \Theta) = \langle \Psi(x; \Theta) | \hat{B}_k | \Psi(x; \Theta) \rangle.$$

Equivalently, with

$$|\Psi(x; \Theta)\rangle = W(\Theta)U(x)|0\rangle,$$

one has

$$f_k(x; \Theta) = \langle 0 | U(x)^\dagger W(\Theta)^\dagger \hat{B}_k W(\Theta) U(x) | 0 \rangle.$$

Commonly $\hat{B}$ is a Pauli operator (e.g. Z on one qubit). In practice one runs many shots on quantum hardware or simulates this circuit classically to estimate $\langle \hat{B}_k \rangle$.

Short summary

In summary, a variational quantum model $f(x; \Theta)$ maps inputs to outputs via the hybrid quantum-classical procedure. During training, the classical optimizer adjusts Θ (e.g. by gradient descent) to minimize a cost function (like mean-squared error) defined on a dataset. Because the mapping is inherently quantum, these models can, in principle, harness the high-dimensional Hilbert space for richer representations. (However, unlike classical deep nets, VQCs may face unique challenges such as gradient vanishing, which we discuss later.)

Mathematical example

For concreteness, consider a 2-qubit circuit. A simple encoding is

$$U(\mathbf{x}) = R_x(x_1) \otimes R_x(x_2),$$

and a variational layer is

$$V(\Theta) = R_y(\Theta_1) \otimes R_y(\Theta_2) \text{CNOT}(0, 1),$$

(apply R_y on each qubit then entangle). After applying $W(\Theta) = V(\Theta)$ to $|00\rangle$, we measure $\hat{B} = Z \otimes I$ on qubit 0. The output is

$$f(\mathbf{x}; \Theta) = \langle 00 | U(\mathbf{x})^\dagger V(\Theta)^\dagger (Z \otimes I) V(\Theta) U(\mathbf{x}) | 00 \rangle.$$

This $f(\mathbf{x}; \Theta)$ is then compared to the target in a cost function for optimization.

Key elements

A VQC is a quantum circuit with trainable parameters acting on a quantum state; it is central to near-term QML (hybrid quantum-classical). Data encoding and ansatz design determine a VQC's expressivity. Simple encodings use rotations (e.g. $R_x(x_i)$) on each qubit, while more complex feature maps may exploit entanglement. The circuit output is obtained via expectation values of observables (e.g. Pauli-Z), yielding a differentiable function $f(x; \Theta)$.

Test yourself exercises

1. Compute the state $|\Psi(x; \Theta)\rangle$ explicitly for a 1-qubit VQC with $U(x) = R_x(x)$ and $W(\Theta) = R_y(\Theta)$. What is $\langle Z \rangle$ as a function of x, Θ ?
2. Draw (or describe) a hardware-efficient ansatz for 3 qubits with 2 layers of rotations and CNOTs. How many parameters does it have?

For the above ansatz, derive the effect of each layer on the state's parameters.

Quantum Neural Networks (QNNs)

Quantum Neural Networks (QNNs) are essentially multi-layer VQCs that mimic classical neural network architectures . One can think of each layer as adding a nonlinear quantum neuron to the network. A simple QNN is a sequence of encoding and variational layers. More structured architectures also exist, such as Quantum Convolutional Neural Networks (QCNNs) and Quantum Long-Short Term Memory networks. In a QCNN, for example, qubits are entangled in a localized pattern to mimic convolution and pooling .

Input Encoding

A crucial aspect of any QNN (as we also saw for QSVMs) is how classical data $x \in \mathbb{R}^d$ are embedded into a quantum state.

Common strategies include:

1. Basis Encoding: Map each bit of x (or feature) to a qubit state $|0\rangle$ or $|1\rangle$. Simple but limited to binary data.
2. Angle (Amplitude) Encoding: Use rotation gates to encode real values, e.g. $R_x(x_i)$ or $R_y(x_i)$ on qubit i .
3. Amplitude Encoding: Embed x into the amplitudes of a multi-qubit state (exponentially compact, but requires complex circuits to prepare).
4. Data Re-uploading: Re-encode input at multiple layers interspersed with trainable gates, effectively increasing expressivity.

The choice of feature map affects performance: no single encoding is best for all tasks. Often one uses a problem-inspired map or random feature circuits, then lets the optimizer adjust the ansatz.

QNN Architecture and Models

A general QNN can be viewed as a parameterized unitary $U(x, \Theta)$ acting on n qubits, followed by measurements. Fig. 2 (placeholder) might depict a generic QNN with several layers of trainable gates. Each layer can entangle qubits, building up complexity. The output is then a (classical) vector of measured values, analogous to the output layer in a classical network.

A simple feedforward QNN structure

1. Embedding Layer: Convert x to $|0\rangle^{\otimes n}$ via $U(x)$.
2. Variational Layers: Repeat L blocks of parameterized gates $W(\Theta^{(l)})$ (each block may act on all or subsets of qubits).
3. Measurement: Measure selected qubits or observables to obtain the output predictions $f(x; \Theta)$.

Example

For example, a 2-layer QNN on 2 qubits might apply encoding $R_x(x_1) \otimes R_x(x_2)$, then apply $W(\Theta^{(1)})$, then again encoding (or not), then $W(\Theta^{(2)})$, and finally measure. In classification tasks, one typically assigns a label based on the sign of $\langle Z \rangle$ or uses multiple measurements for multi-class outputs.

Notably, even though QNNs operate on exponentially large Hilbert spaces, their actual power is subject of research. Abbas et al. introduce the notion of effective dimension and argue that some QNNs can outperform classical networks in terms of trainability and generalization . However, other studies point out that QNNs may suffer from trainability issues.

Training output and Cost/Loss-function

Given a QNN with output $f(x; \Theta)$ (a real number or vector of real values), one must define a loss function to train on data. Common choices are the mean squared error (MSE) for regression or cross-entropy for classification. For a training set x_i, y_i , the MSE cost/loss-function is

$$C(\Theta) = \frac{1}{N} \sum_{i=1}^N (f(x_i; \Theta) - y_i)^2.$$

One then computes gradients $\nabla_{\Theta} C$ and updates parameters via gradient descent or other optimizers.

Example: Variational Classifier

A binary classifier can output $f(x; \Theta) = \langle Z_0 \rangle$ on qubit 0, and predict label $+1$ if $f \geq 0$, else -1 .

Variational Layer Algebra

As a warm-up problem, consider two qubits with single-qubit rotations $R_y(\alpha)$ on each qubit followed by a CNOT. Show that this two-qubit gate can create entanglement if α is not a multiple of π . (Hint: apply it to $|00\rangle$ and compute the resulting state.) This demonstrates how trainable gates can correlate qubits, enriching the model.

Short summary

A QNN is implemented by layering VQCs; it generalizes neural networks to quantum circuits . Encoding maps classical features to quantum states (e.g. via rotation gates). The ansatz (variational layers) defines the network's expressive power; depth and entanglement matter. Output is given by expectation(s) of measured observables, which are compared against targets via a classical loss function.

Training QNNs and Loss Landscapes

Training a QNN involves optimizing a non-convex quantum circuit cost function. Like classical neural networks, one typically uses gradient-based methods. However, VQCs have unique features, as listed here.

Gradient Computation

Gradients $\partial f / \partial \Theta_j$ are obtained using the parameter-shift rule. For many gates $e^{-i\Theta P/2}$ (with P a Pauli), one can compute

$$\frac{\partial}{\partial \Theta} \langle B \rangle = \frac{1}{2} \left[\langle B \rangle_{\Theta + \pi/2} - \langle B \rangle_{\Theta - \pi/2} \right],$$

where $\langle B \rangle_{\Theta \pm \pi/2}$ are expectation values evaluated at shifted parameter values. This formula allows exact gradients by two circuit evaluations per parameter (independent of circuit size). PennyLane automatically applies parameter-shift rule when you call backward on a QNode . Optimizers: One can use gradient descent or more advanced optimizers (Adam, ADAgrad, RMSprop, etc.). PennyLane provides a `qml.GradientDescentOptimizer` and others. Gradients flow through the classical loss into the quantum circuit via the parameter-shift trick. In our code examples below we will see this in action.

Barren Plateaus

A major challenge is the barren plateau phenomenon . In deep or highly entangled circuits, the loss landscape can become extremely flat: gradients vanish exponentially with system size. As Anschuetz and Kiani note, variational models often become untrainable due to vanishing gradients in deep layers . Even surprisingly, their work shows that shallow circuits may still have very few “good” local minima near the global optimum . In practice, this means random initialization of a deep QNN often leads to tiny gradients, stalling training. Mitigation Strategies: Researchers propose various remedies to avoid or alleviate barren plateaus. Examples include layerwise training (training a few layers at a time), smart initialization (e.g. initializing most gates to identity), and ansatz design (using problem-inspired or shallow circuits to avoid global entanglement). Another approach uses local cost functions: measuring local observables rather than global ones can reduce gradient concentration. These strategies are active research areas, but remain crucial for making QNN training feasible on near-term devices.

Cost/Loss-landscape visualization

One can imagine the cost/loss function $C(\Theta)$ over the parameter space. Unlike convex classical problems, this landscape may have many local minima and saddle points. Barren plateaus correspond to regions where $\nabla C \approx 0$ almost everywhere. Even if plateaus are avoided, poor minima can still trap the optimizer. In practice, careful tuning of learning rates and adding small random noise can help escape shallow minima.

QNN training uses classical optimizers on circuit outputs, with gradients given by the parameter-shift rule. Barren plateaus (vanishing gradients) are a central obstacle in deep circuits. Mitigation includes shallow ansatz, structured circuits, and smart initialization. Always monitor training and consider multiple random restarts to find good minima.

Exercises

1. Compute a gradient by hand: For a circuit with one qubit and $f(\Theta) = \langle 0 | R_y(\Theta)^\dagger Z R_y(\Theta) | 0 \rangle$, use the parameter-shift rule to compute $df/d\Theta$.
2. Explore barren plateaus: Numerically evaluate $\partial f / \partial \Theta$ for a simple 5-qubit random circuit as depth increases. Observe the trend of gradient norms. What does this suggest?
3. Optimizer effects: Implement a small QNN (2 qubits) and train with both SGD and Adam optimizers. Compare convergence speed.

Implementing QNNs with PennyLane

PennyLane provides QNodes, differentiable quantum functions that can be integrated with Python ML frameworks. Here we illustrate building and training a simple variational quantum classifier using PennyLane.

```
import pennylane as qml
from pennylane import numpy as np

# Create a 2-qubit simulator device
dev = qml.device('default.qubit', wires=2)

# Define a feature map (state preparation) circuit
def feature_map(x):
    qml.RX(x[0], wires=0)
    qml.RX(x[1], wires=1)

# Define a variational (trainable) layer
def variational_layer(params):
    # params is a list of 4 angles for 2 qubits
    qml.Rot(params[0], params[1], params[2], wires=0)
    qml.Rot(params[3], params[0], params[1], wires=1)
    qml.CNOT(wires=[0,1])

# Define the QNode: quantum classifier circuit
@qml.qnode(dev)
def qclassifier(params, x=None):
    # encode data into quantum state
    feature_map(x)
```

Using PennyLane

PennyLane's `qml.qnode` decorator converts a quantum circuit into a function whose gradients can be computed automatically . We combine the feature map and variational layers inside a single `QNode` to form a model. The cost function compares the QNN's output to labels; optimization is done classically. PennyLane allows seamless mixing of quantum nodes and classical Python code, facilitating experimentation.

Additional exercises

1. Modify the above code to use `qml.AdamOptimizer` and compare training convergence.
2. Extend the circuit by adding a third qubit (use `wires=3`) and corresponding rotations. How does this affect the model's capacity?
3. Implement a simple dataset (e.g. points arranged in XOR pattern) and train the QNN. Evaluate its classification accuracy.

Variational Quantum Neural Network for credit classification

This self-contained PennyLane code demonstrates a simple hybrid quantum-classical binary classifier on synthetic financial data, like the one we discussed in connection with quantum support vector machines.

We build a quantum neural network (QNN) – a parameterized (variational) quantum circuit – to classify synthetic credit data. Quantum neural networks are typically implemented as variational quantum circuits with trainable rotation angles.

We first generate small synthetic data with features like income, debt ratio, and age, labeling each datapoint as “good” or “bad” credit. Classical features are encoded into qubit rotation angles using an angle embedding, and a layer of trainable entangling gates (PennyLane’s `StronglyEntanglingLayers`) forms the variational ansatz. During training, we optimize the circuit parameters via a classical optimizer to minimize binary cross-entropy loss . Finally, we measure one qubit to produce a probability for the positive class and evaluate the classifier with accuracy, precision, and recall . The following code implements all steps using PennyLane’s default.qubit simulator

Essential steps in the code

Data Encoding:

We map each feature vector to quantum states via angle embedding: each feature is used as the rotation angle of an RY gate on a qubit . This creates a **quantum feature map** of our classical data.

Variational Ansatz:

After embedding, we apply a layer of trainable rotations and entangling gates (StronglyEntanglingLayers), creating a parameterized circuit whose outputs depend on adjustable weights . Measuring the expectation $\langle Z \rangle$ of the first qubit yields a value in $[-1, 1]$, which we convert to a class probability via $(1 + \langle Z \rangle)/2$.

Training:

We optimize the circuit parameters by minimizing the binary cross-entropy loss between the predicted probabilities and true labels. Binary cross-entropy (log-loss) is a standard choice for binary classification , adjusting weights to improve the match between predictions and targets. We use PennyLane's

Applications and examples

Quantum neural networks and VQCs have been explored in various contexts. Here we briefly mention some notable examples (the field is rapidly growing):

Quantum Classification and Regression:

As demonstrated above, QNNs can classify classical data points by learning decision boundaries in Hilbert space . Extensions include multiclass classification, embedding kernels, and combining classical-quantum pipelines.

Quantum Approximate Optimization Algorithm (QAOA):

Though not a neural network per se, QAOA is a VQA for combinatorial optimization. It can be seen as a special case where the Hamiltonian objective is minimized by a parameterized circuit. PennyLane supports QAOA out of the box.

Additional applications and examples

Variational Quantum Eigensolver (VQE):

Another hybrid algorithm for finding ground-state energies; useful in quantum chemistry. While not ML, it uses similar VQC training techniques.

Quantum Generative Models:

Variational circuits can be trained to produce quantum states resembling a target distribution (Quantum GANs, Quantum Boltzmann Machines). This is an active research area.

Quantum Reinforcement Learning:

Researchers have proposed using QNNs as function approximators (policy or value functions) in RL. Some works embed classical observations into quantum states and train QNNs by classical RL algorithms.

More on applications

Quantum Kernel Methods:

Instead of a neural net, one can use VQCs to define kernels for classical kernel machines. PennyLane provides modules for quantum kernel evaluation.

Hardware Demonstrations:

Small-scale QML experiments have been run on IBM, Google, and IonQ devices. These serve as proof-of-concept for the hybrid model.

References of interest

1. A. Abbas et al., **The power of quantum neural networks**, Nature Comput. Sci. 1, 403–409 (2021).
2. E. Anschuetz and B. Kiani, **Quantum variational algorithms are swamped with traps**, Nat. Commun. 13, 7760 (2022) .
3. J. Millet et al., **Variational quantum circuits for machine learning: an application for detecting weak signals**, Appl. Sci. 11, 6427 (2021) .
4. M. Zhao et al., **A tutorial on quantum machine learning and quantum neural networks**, arXiv:2504.16131 (2025) .
5. D. M. Houshmand, **Quantum Machine Learning Lecture Notes**, (online, 2023) . (Tutorial using PennyLane QNNs.)
6. PennyLane documentation and tutorials:
<https://pennylane.a>) (for code examples and parameter-shift rules).